L08: Appreciating Low Rank Data

C. V. Jawahar

IIIT Hyderabad

February 8, 2026

Recap:

- Problems of interest:
  - Learn a function $y = f(\mathbf{W}, \mathbf{x})$ from the data.
    - (a) Classification (b) Regression
  - Learn Feature Transformations $\mathbf{x}' = \mathbf{W}\mathbf{x}$ or $\mathbf{x}' = f(\mathbf{W}, \mathbf{x})$
    - (a) Feature Normalization (b) PCA/Dimensionality Reduction
- Algorithms/Approaches:
  - (i) Nearest Neighbour Algorithm (ii) Linear Classification: $sign(\mathbf{w}^T\mathbf{x})$ (iii) Decide as ω_1 if $P(\omega_1|\mathbf{x}) \geq P(\omega_2|\mathbf{x})$ else ω_2.
  - Linear Regression: (a) closed form and (b) GD; Ridge and Lasso
- Supervised Learning:
  - Notion of Training, Validation and Testing, Performance Metrics
  - Low rank data matrix and SVD; PCA and Eigen Faces
  - Notion of Loss Function, (eg. MSE), Regularization.
  - Role of Optimization, Convex and non-Convex optimization
  - Closed form solution, Gradient Descent, Eigen vector solns.

PCA: Algorithm

1. Input: N samples of D dimension
2. Compute Covariance Matrix $$\Sigma = \frac{1}{N}\sum_{i=1}^{N}[x_i - \mu][x_i - \mu]^T$$
3. Compute Eigen Values and Eigen Vectors of Covariance Matrix
4. Select `d' eigen vectors corresponding to 'd' largest eigen values
5. Arrange them as rows of 'A' (a 'd' X 'D' matrix)
6. Find lower dimensional representations as: $x' = Ax$

Eigen face (Recap)

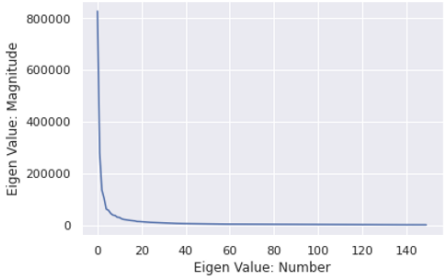
Eigen Faces

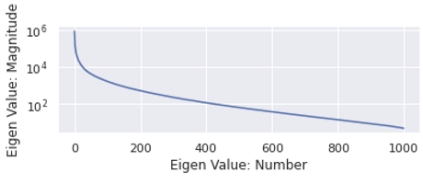
62 X 47 Image D = 2914; N = 1348
Vectors displayed as 2D Image

Mean
2914 X 1

Covariance Matrix is
2914 X 2914

First 16 Eigen Vectors
out of 2914





Log-scale

https://towardsdatascience.com/eigenfaces-recovering-humans-from-ghosts-17606c328184

Note on Reconstruction

Consider a sample in two coordinate systems. Let $\mathbf{x} = [x^1, x^2, \ldots, x^d]^T$

1. Standard basis: $\mathbf{u_1}, \mathbf{u_2}, \ldots \mathbf{u_d}$
   - $\mathbf{u_i} = [0, 0, \ldots, 1, \ldots, 0, 0,]^T$. WIth '1' at the i th place
   - $\mathbf{x} = (\mathbf{x}^T\mathbf{u}_1)\mathbf{u}_1 + (\mathbf{x}^T\mathbf{u}_2)\mathbf{u}_2 + \ldots + (\mathbf{x}^T\mathbf{u}_d)\mathbf{u}_d$
   - $\mathbf{x} = x^1\mathbf{u}_1 + x^2\mathbf{u}_2 + \ldots + x^d\mathbf{u}_d$
2. Defined by Eigen Vectors: $\mathbf{v_1}, \mathbf{v_2}, \ldots, \mathbf{v_d}$
   - $(\lambda_i, \mathbf{v}_i)$ are the eigen value and eigen vectors of the covariance matrix.
   - $\mathbf{x} = (\mathbf{x}^T\mathbf{v}_1)\mathbf{v}_1 + (\mathbf{x}^T\mathbf{v}_2)\mathbf{v}_2 + \ldots + (\mathbf{x}^T\mathbf{v}_{d'})\mathbf{v}_{d'} + \ldots + (\mathbf{x}^T\mathbf{v}_d)\mathbf{v}_d$
   - $\mathbf{x} = \alpha_1\mathbf{v}_1 + \alpha_2\mathbf{v}_2 + \ldots + \alpha_{d'}\mathbf{v}_{d'} + \ldots + \alpha_d\mathbf{v}_d$

If we reduce to d' dimension from d, we loose some information/signal.

PCA as Minimizing Reconstructing Loss

Let $\mathbf{u_1}, \ldots \mathbf{u_d}$ be d orthonormal vectors. $\mathbf{x}$ is $\sum_{i=1}^{d} \alpha_i \mathbf{u}_i$, where $\alpha_i = \mathbf{x}^T \mathbf{u_i}$. Problem: Find the dimension $\mathbf{u}$ that has highest variance (or minimal reconstruction error) or minimize $\mathbf{x} - \mathbf{u}\mathbf{u}^T\mathbf{x}$. Problem is to minimze:

$$\sum_{i=1}^{N} ||\mathbf{x_i} - \mathbf{u}\mathbf{u}^T\mathbf{x}_i||^2 = \sum_{i=1}^{N} \left(\mathbf{x}_i^T\mathbf{x}_i + (\mathbf{u}\mathbf{u}^T\mathbf{x}_i)^T(\mathbf{u}\mathbf{u}^T\mathbf{x}_i) - 2\mathbf{x}_i^T\mathbf{u}\mathbf{u}^T\mathbf{x}_i \right)$$

First term is positive and independent of $\mathbf{u}$. And $\mathbf{u}^T\mathbf{u} = 1$. Minimize:

$$\sum_{i=1}^{N} (\mathbf{x_i}^T\mathbf{u}\mathbf{u}^T\mathbf{u}\mathbf{u}^T\mathbf{x_i} - 2\mathbf{x}_i^T\mathbf{u}\mathbf{u}^T\mathbf{x}_i) = \sum_{i=1}^{N} -\mathbf{x}_i^T\mathbf{u}\mathbf{u}^T\mathbf{x}_i = \sum_{i=1}^{N} -\mathbf{u}^T\mathbf{x}_i\mathbf{x}_i^T\mathbf{u} = -\mathbf{u}^T\Sigma\mathbf{u}$$

Thus the problem reduces to:

$$\text{Maximize } \mathbf{u}^T\Sigma\mathbf{u} \text{ such that } \mathbf{u}^T\mathbf{u} = 1$$

This reduces the solution as the eigen vectors corresponding to the largest eigen values.

Dimensionality Reduction

Reduce dimensionality

- Linear:$\mathbf{x}' = \mathbf{W}\mathbf{x}$ (eg. PCA)
- Nonlinear $\mathbf{x}' = f(\mathbf{W}, \mathbf{x})$ (eg. Autoencoder neural networks)
- Supervised (LDA, Fisher)
- Unsupervised (PCA, autoencoder)
- Dimensionality Reduction on a Manifold (LLE, ISOMAP) (out of scope)

Why Dimensionality Reduction? Many Reasons ..

- Data compression
- Less parameters to learn (less data needed)
- Less overfittiong (More reliable models/solutions)
- Faster computation
- Better visualization

Why Dimensionality Reduction? Separability

Fisher Linear Discriminant or LDA Goal (two classes):

- Classes are well separated (means are far) (Maximimize S_B)
- Each class is compact (variance is small) (Minimize S_W)

$$S_B = [\mu_1 - \mu_2][\mu_1 - \mu_2]^T$$

$$S_W = S_1 + S_2 = \sum_{\mathbf{x_i} \in \omega_1} [\mathbf{x_i} - \mu_1][\mathbf{x_i} - \mu_1]^T + \sum_{\mathbf{x_i} \in \omega_2} [\mathbf{x_i} - \mu_2][\mathbf{x_i} - \mu_2]^T$$

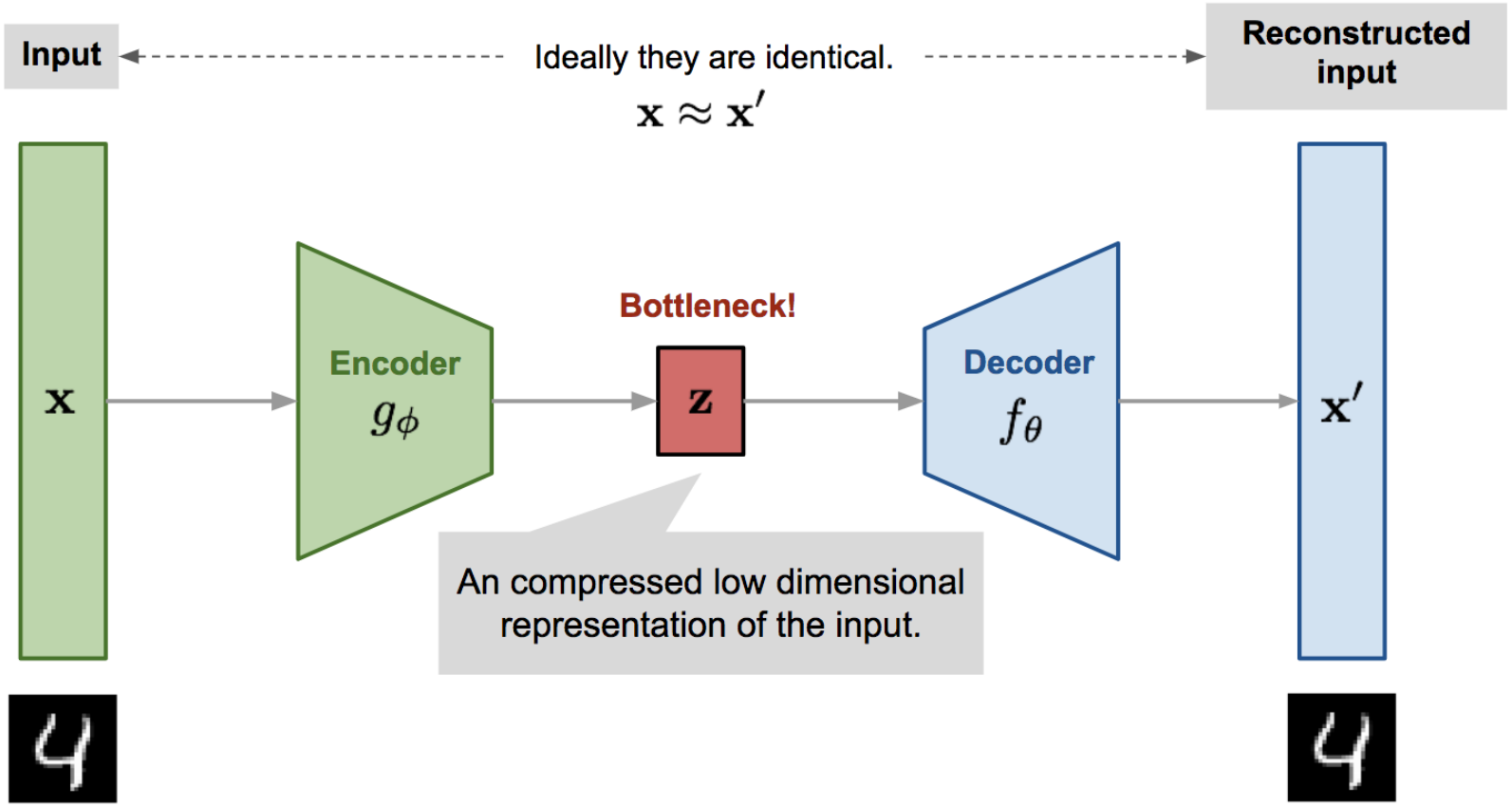
If we reduce $z = \mathbf{u}^T\mathbf{x}$ our goal is to maximize: $J(\mathbf{u}) = \frac{\mathbf{u}^T\mathbf{S_B}\mathbf{u}}{\mathbf{u}^T\mathbf{S_W}\mathbf{u}}$

$$\max_{\mathbf{u}} \frac{1}{2}\mathbf{u}^T\mathbf{S_B}\mathbf{u} - \frac{\lambda}{2}(\mathbf{u}^T\mathbf{S_W}\mathbf{u} - 1)$$

Leads to a genarlized eigen value problem $\mathbf{S_B}\mathbf{u} = \lambda\mathbf{S_W}\mathbf{u}$ with solution as

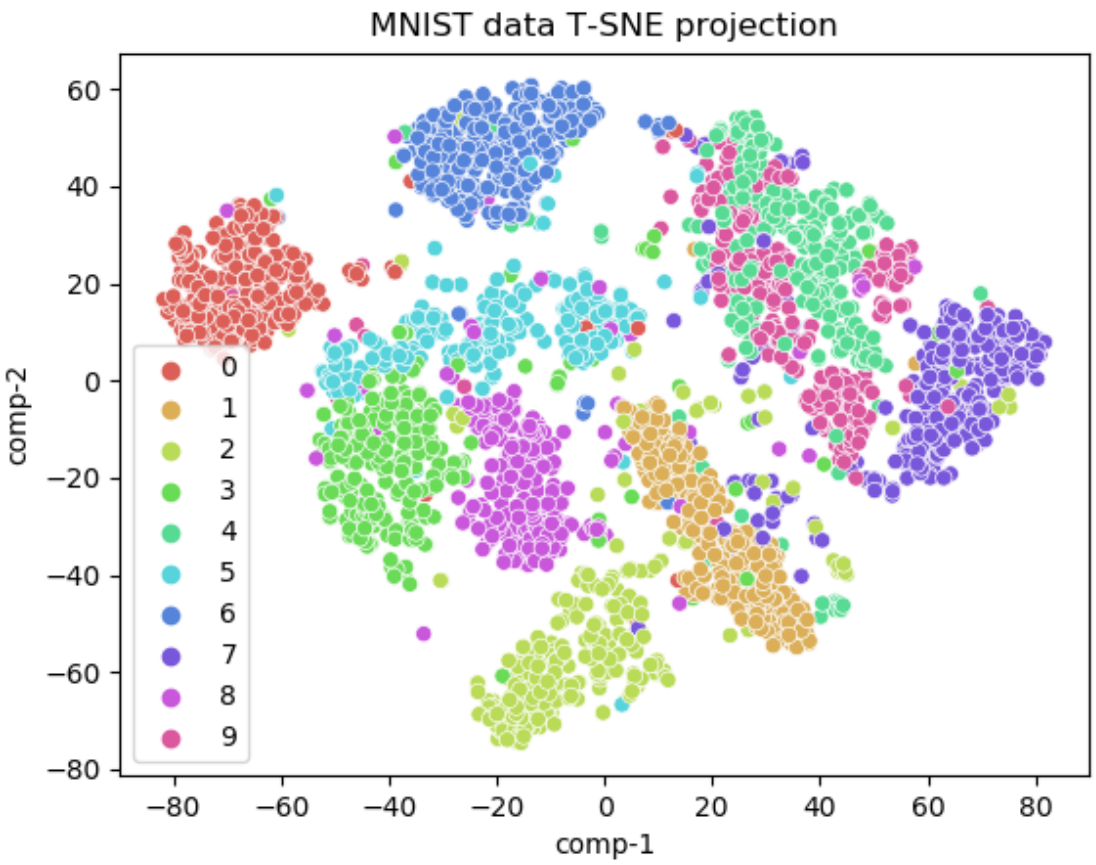
$$\mathbf{u} = \alpha\mathbf{S_W}^{-1}[\mu_1 - \mu_2]$$

Non-Linear Dimensionality Reduction: Auto Encoder



An excellent example of unsupervised learning

Why Dimensionality Reduction? Visualization: tsne[1]



[1] https://en.wikipedia.org/wiki/T-distributed_stochastic_neighbor_embedding

Why Dimensionality Reduction? TSNE

t-distributed stochastic neighbor embedding

Characteristic	t-SNE	PCA
Type	Non-linear dimensionality reduction	Linear dimensionality reduction
Goal	Preserve local pairwise similarities	Preserve global variance
Best used for	Visualizing complex, high-dimensional data	Data with linear structure
Output	Low-dimensional representation	Principal components
Use cases	Clustering, anomaly detection, NLP	Noise reduction, feature extraction
Computational intensity	High	Low
Interpretation	Harder to interpret	Easier to interpret

Eigen Decomposition

Popular eigen expansion

$$\Sigma = \sum_{i=1}^{d} \lambda_i \mathbf{v}_i \mathbf{v}_i^T$$

tells us what variation to retain and what to approximate.

$$\Sigma = \sum_{i=1}^{d'} \lambda_i \mathbf{v}_i \mathbf{v}_i^T + \sum_{i=d'+1}^{d} \lambda_i \mathbf{v}_i \mathbf{v}_i^T$$

We chose the reduced dimension based on the eigen value spectrum.
Also for a general PSD matrix

$$A = U \Lambda U^T$$

is known as the eigen decomposition.

SVD

Formally, the **singular value decomposition** (SVD) of an $m \times n$ real or complex matrix M is a factorization of the form

$$M = UDV^T$$

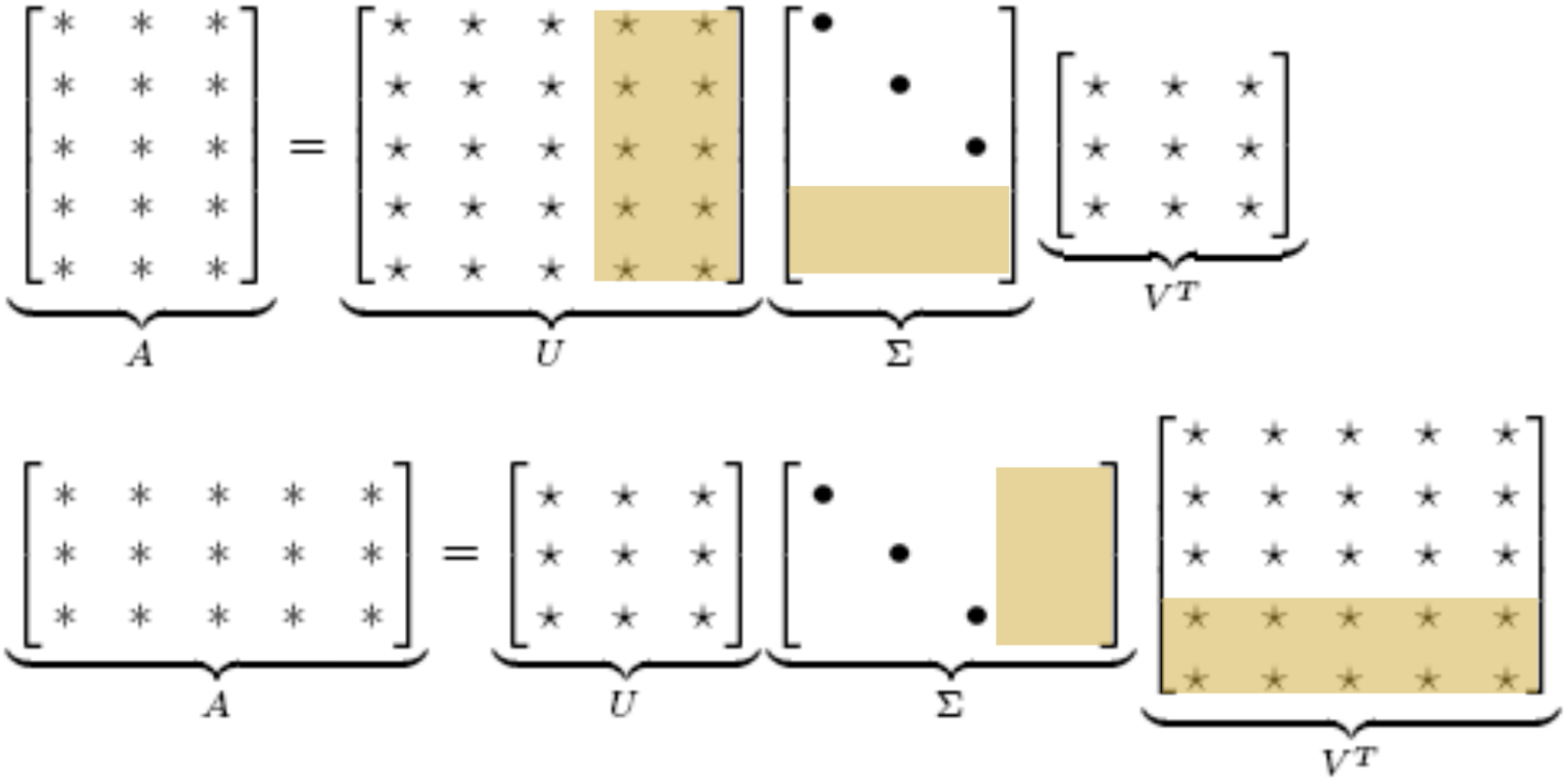
- U is $m \times n$, $U^T U = I_n$,
- D is diagonal $n \times n$ and
- V is $V^T V = VV^T = I_n$.

$$M = \sum_i D_i u_i v_i^T$$

Read this article[2]

[2] https://jonathan-hui.medium.com/machine-learning-singular-value-decomposition-svd-principal-component-anal

SVD



Eigen vs SVD

We know that:

- Eigen Decomposition of Symmetric Matrix **S**

$$\mathbf{S} = \mathbf{Q}\Lambda\mathbf{Q}^T = \sum_{i=1}^{n} \lambda_i \mathbf{q}_i \mathbf{q}_i^T$$

- SVD of **A**

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T = \sum_{i=1}^{n} d_i \mathbf{u}_i \mathbf{v}_i^T$$

- The non-zero singular values of M (found on the diagonal entries of D) are the square roots of the non-zero eigenvalues of both $M^T M$ and MM^T.

Low Rank Approximation using SVD

Problem: Given a matrix **A**, find its low rank (k) approximation.

1. Find SVD of **A** as $A \rightarrow UDV^T$
2. Make all singular values beyond k in D as zero to obtain $\hat{D}$
3. Reconstruct k rank matrix

$$\hat{A} \leftarrow U\hat{D}V^T$$

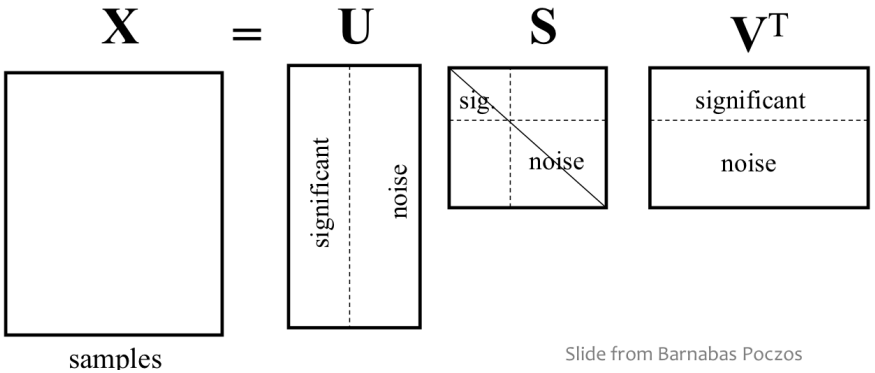
4. Or

$$\hat{A} \leftarrow U_k D_k V_k^T$$

where U_k and V_k matrices are the first k columns

SVD and PCA

$$\mathbf{X}_{\text{features} \times \text{samples}} = \mathbf{USV}^T$$



Slide from Barnabas Poczos

- **Columns of U**
  - the principal vectors, $\{ \mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(k)} \}$
  - orthogonal and has unit norm – so $U^T U = I$
  - Can reconstruct the data using linear combinations of $\{ \mathbf{u}^{(1)}, \ldots, \mathbf{u}^{(k)} \}$

- **Matrix S**
  - Diagonal
  - Shows importance of each eigenvector

- **Columns of V^T**
  - The coefficients for reconstructing the samples

Matrix Problems

- Matrix Factorization/Decomposition
- Non-Negative Matrix factorization
- Matrix Completion
- Low rank Approximation

Discussions Point (Recap)

Matrix completion: We know completion of rank 1 matrix as:

$$A = \begin{bmatrix} 7 & ? & ? \\ ? & 8 & ? \\ ? & 12 & 6 \\ ? & ? & 2 \\ 21 & 6 & ? \end{bmatrix} \Rightarrow \begin{bmatrix} 7 & 2 & 1 \\ 28 & 8 & 4 \\ 42 & 12 & 6 \\ 14 & 4 & 2 \\ 21 & 6 & 3 \end{bmatrix}$$

Example Problem Formulation:

$$\min \sum_i \sum_j (A_{ij} - B_{ij})^2 \ s.t \ rank(B) = 1$$

Q: What is this summation over? What is the min over?

The rank constraint is not easy to enforce. See alternate formulations based on Nuclear norm[3]. i.e., L1 norm over Diag(D) instead of L0 norm.

[3] Nuclear norm of a matrix is just the sum of the singular values of the matrix

Applications: Watch Later

- LSI Watch (10 mins):
  - https://www.youtube.com/watch?v=rMaDnRLZAzM
- Recommendation Systems Watch (10 mins):
  - https://www.youtube.com/watch?v=tP9e0PlmMKo

Examples of Low Rank Matrices

User-Rating

- User Vs Products (1 or 0)
- User Vs Movies (say Rating in range 1, to 5

Term Document Matrix

- A matrix of Tems Vs Documents (0 or 1 ; or frequency)
- Say the documents are either medical or legal

Questions:

1. Are these often sparse or dense matrices?
2. Are these low-Rank matrices?
3. How do we recommend products?
4. How do we complete matrices?

[4]

[4] Watch `https://www.youtube.com/watch?v=tP9e0PlmMKo`

Latent Semantic Indexing (LSI)

Consider a term document matrix ($N \times d$) where there are N documents and d is the vocabulary size. We argued that this matrix is often low-rank in practice.

Consider a collection of documents from "Technology" and "Food".

- How many "concepts" topics are there? How do we find them?
- How do we handle words like "apple" which may have multiple meaning?

LSI

C	d_1	d_2	d_3	d_4	d_5	d_6
ship	1	0	1	0	0	0
boat	0	1	0	0	0	0
ocean	1	1	0	0	0	0
wood	1	0	0	1	1	0
tree	0	0	0	1	0	1

U	1	2	3	4	5
ship	−0.44	−0.30	0.00	0.00	0.00
boat	−0.13	−0.33	0.00	0.00	0.00
ocean	−0.48	−0.51	0.00	0.00	0.00
wood	−0.70	0.35	0.00	0.00	0.00
tree	−0.26	0.65	0.00	0.00	0.00

V^T	d_1	d_2	d_3	d_4	d_5	d_6
1	−0.75	−0.28	−0.20	−0.45	−0.33	−0.12
2	−0.29	−0.53	−0.19	0.63	0.22	0.41
3	0.00	0.00	0.00	0.00	0.00	0.00
4	0.00	0.00	0.00	0.00	0.00	0.00
5	0.00	0.00	0.00	0.00	0.00	0.00

Σ	1	2	3	4	5
1	2.16	0.00	0.00	0.00	0.00
2	0.00	1.59	0.00	0.00	0.00
3	0.00	0.00	1.28	0.00	0.00
4	0.00	0.00	0.00	1.00	0.00
5	0.00	0.00	0.00	0.00	0.39

Σ_2	1	2	3	4	5
1	2.16	0.00	0.00	0.00	0.00
2	0.00	1.59	0.00	0.00	0.00
3	0.00	0.00	0.00	0.00	0.00
4	0.00	0.00	0.00	0.00	0.00
5	0.00	0.00	0.00	0.00	0.00

LSI: Why and How LSI works?

C	d_1	d_2	d_3	d_4	d_5	d_6
ship	1	0	1	0	0	0
boat	0	1	0	0	0	0
ocean	1	1	0	0	0	0
wood	1	0	0	1	1	0
tree	0	0	0	1	0	1

C_2	d_1	d_2	d_3	d_4	d_5	d_6
ship	0.85	0.52	0.28	0.13	0.21	−0.08
boat	0.36	0.36	0.16	−0.20	−0.02	−0.18
ocean	1.01	0.72	0.36	−0.04	0.16	−0.21
wood	0.97	0.12	0.20	1.03	0.62	0.41
tree	0.12	−0.39	−0.08	0.90	0.41	0.49

- How close are document d_2 and d_3 in the original and new space?

Problem

(based on the LSI example) We have a large number of documents. They are represented in a Term Document Matrix. Given a query document, find the most semantically similar document. Given matrix **A** is a term document matrix (with columns as document d_1, d_2, d_3 etc.). Given a new document as $[1, 0, 1, 0, 0]^T$, find the cosine similarity with all the six documents and find the nearest document
in the latent space of dimension 2 as well as 1.

Hint: map query and documents as $d^T U_k D_K^{-1}$

Problem 1

If $\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^{\mathbf{T}}$, Find the SVD of $\mathbf{A}^{-1}$.

Problem 2

1. What is left and right inverse of a matrix? How do we compute using SVD?

Problem 3

If $\mathbf{A} = UDV^T$.

- What are the eigen values and eigen vectors of AA^T?
- What are the eigen values and eigen vectors of A^TA?

Problem 4

Consider a matrix A formed by first 25 integers in row order $1, \ldots, 25$. Find a rank-1 and rank-2 approximation of A. What is the error in this approximation? (use an online SVD tool for computing SVD)

Review Questions

1. If $A = UDV^T$, then $A^T A$ is
   - (i) always a rank deficient matrix (ii) VD^2V^T (iii) UD^2U^T (iv) A square matrix (v) is always full rank (vi) can some times be full rank (vi) none of the above etc.
2. Consider a set of general vectors $\mathbf{a}_i \in R^d$. (assume all elements are some random numbers in the range of $[0, 1]$) $\mathbf{b}$ is another such vector. Consider the matrix:

$$\mathbf{A} = \sum_{i=1}^{k} \alpha_i \mathbf{a}_i \mathbf{a}_i^T + \sum_{i=k+1}^{d} \beta_i \mathbf{b}\mathbf{b}^T$$

Comment on the effective rank of $\mathbf{A}$ for various values of α_i and β_i

Discussion Points

LSI has many applications [5]

- Compare the documents in the low-dimensional space (data clustering, document classification).
- Find similar documents across languages, after analyzing a base set of translated documents (cross-language information retrieval).
- Find relations between terms (synonymy and polysemy).
- Given a query of terms, translate it into the low-dimensional space, and find matching documents (information retrieval).
- Find the best similarity between small groups of terms, in a semantic way.
- Expand the feature space of machine learning / text mining systems
- Analyze word association in text corpus

Q: **How does it help in Synonymy** ?

Q: **How does it help in Clustering documents** ?

[5] Read: https://en.wikipedia.org/wiki/Latent_semantic_analysis